

RTL Synthesis and Gate-Level Simulation of a Traffic Light Controller

Project Report

Sr. No.	INDEX	
1.	Abstract	
2.	Introduction	
3.	Objectives	
4.	Software and Tools Used	
5.	Design Description	
6.	RTL Code	
7.	Testbench	
8.	Simulation was performed using Vivado Simulator	
9.	RTL Synthesis	
10.	Gate-Level Netlist Generation	
11.	Gate-Level Simulation	
12.	Static Timing Analysis (STA)	
13.	Results	
14.	Applications	
15.	Advantages	
16.	Conclusion	
17.	References	

RTL Synthesis and Gate-Level Simulation of a Traffic Light Controller

Abstract

This project presents the design and implementation of a Traffic Light Controller using Verilog HDL. The design was simulated, synthesized, and verified using Xilinx Vivado. RTL simulation was performed to validate the functionality of the controller. The RTL design was then synthesized into a gate-level netlist, followed by gate-level simulation and static timing analysis to ensure correct operation and timing compliance.

Introduction

Traffic Light Controllers are widely used in road intersections to regulate vehicle movement and improve traffic safety. Digital controllers are commonly implemented using Finite State Machines (FSMs) that switch between RED, GREEN, and YELLOW lights in a predefined sequence.

This project demonstrates the design flow from RTL coding to gate-level verification using industry-standard FPGA design tools.

Objectives

- Design a Traffic Light Controller using Verilog HDL.
- Verify functionality through RTL simulation.
- Synthesize the RTL design into a gate-level netlist.
- Perform gate-level simulation.
- Conduct Static Timing Analysis (STA).
- Analyze synthesis and timing reports.

RTL Synthesis and Gate-Level Simulation of a Traffic Light Controller

Software and Tools Used

Tool	Purpose
Xilinx Vivado	Design Entry, Synthesis and Simulation
Verilog HDL	Hardware Description Language
Vivado Simulator	RTL and Gate-Level Simulation

Design Description

The Traffic Light Controller operates using three states:

1. RED Light
2. GREEN Light
3. YELLOW Light

The controller cycles through these states continuously on each clock pulse.

Inputs

- clk : Clock Signal
- rst : Reset Signal

Outputs

- light[2:0] : Traffic Light Output

Light	Output
-------	--------

RED	100
-----	-----

RTL Synthesis and Gate-Level Simulation of a Traffic Light Controller

GREEN 001

YELLOW 010

RTL Code

```
module traffic_light(  
    input clk,  
    input rst,  
    output reg [2:0] light  
);  
  
parameter RED    = 3'b100;  
parameter GREEN  = 3'b001;  
parameter YELLOW = 3'b010;  
  
reg [1:0] state;  
  
always @(posedge clk or posedge rst)  
begin  
    if(rst)  
        begin  
            state <= 2'b00;  
            light <= RED;  
        end  
    else
```

RTL Synthesis and Gate-Level Simulation of a Traffic Light Controller

```
begin
    case(state)
        2'b00:
            begin
                light <= RED;
                state <= 2'b01;
            end
        2'b01:
            begin
                light <= GREEN;
                state <= 2'b10;
            end
        2'b10:
            begin
                light <= YELLOW;
                state <= 2'b00;
            end
        default:
            begin
                light <= RED;
                state <= 2'b00;
            end
    end
```

RTL Synthesis and Gate-Level Simulation of a Traffic Light Controller

```
        endcase  
    end  
end  
endmodule
```

Testbench

```
`timescale 1ns/1ps  
module tb_traffic;  
    reg clk;  
    reg rst;  
    wire [2:0] light;  
    traffic_light uut(  
        .clk(clk),  
        .rst(rst),  
        .light(light)  
    );  
    always #5 clk = ~clk;  
    initial begin  
        clk = 0;  
        rst = 1;
```

RTL Synthesis and Gate-Level Simulation of a Traffic Light Controller

```
#20 rst = 0;  
  
#100;  
  
$finish;  
  
end  
  
endmodule
```

Simulation was performed using Vivado Simulator.

Expected Output Sequence

Clock Cycle Output Light

1	100	RED
2	001	GREEN
3	010	YELLOW
4	100	RED
5	001	GREEN

The waveform confirmed correct state transitions.

RTL Synthesis

The Verilog RTL design was synthesized using Vivado.

Synthesis Procedure

1. Create a New Vivado Project.
2. Add RTL Source Files.
3. Add Constraint File.

RTL Synthesis and Gate-Level Simulation of a Traffic Light Controller

4. Run Synthesis.
5. Open Synthesized Design.
6. View Schematic.

The synthesis process converted the FSM into FPGA resources such as LUTs and flip-flops.

Gate-Level Netlist Generation

After synthesis, the gate-level netlist was generated using the Tcl command:

```
write_verilog -mode funcsim traffic_netlist.v
```

The generated file contains the synthesized logic representation of the Traffic Light Controller.

Gate-Level Simulation

Post-synthesis functional simulation was performed using the generated gate-level netlist.

Procedure

1. Run Post-Synthesis Functional Simulation.
2. Load the synthesized netlist.
3. Execute simulation.
4. Compare results with RTL simulation.

Result

The gate-level simulation produced the same traffic light sequence as the RTL simulation.

RTL Synthesis and Gate-Level Simulation of a Traffic Light Controller

Static Timing Analysis (STA)

Timing constraints were applied using an XDC file.

Clock Constraint

```
create_clock -period 10 [get_ports clk]
```

Clock Frequency:

100 MHz

Timing Summary

Parameter	Value
Clock Period	10 ns
WNS	Positive
TNS	0 ns

Setup Violations 0

Hold Violations 0

The design met all timing requirements.

Results

- RTL simulation executed successfully.
- Traffic light sequencing was verified.
- Synthesis completed without errors.
- Gate-level netlist was generated successfully.
- Gate-level simulation matched RTL behavior.
- Timing analysis reported no violations.

RTL Synthesis and Gate-Level Simulation of a Traffic Light Controller

Applications

- Smart Traffic Signal Systems
- Road Intersection Control
- Transportation Management Systems
- FPGA-Based Embedded Controllers
- Intelligent Traffic Monitoring Systems

Advantages

- Simple FSM-based design
- Easy implementation on FPGA
- Low hardware resource utilization
- Reliable operation
- Expandable for multi-road intersections

Conclusion

A Traffic Light Controller was successfully designed using Verilog HDL and implemented using Xilinx Vivado. The design was verified through RTL simulation, synthesized into a gate-level netlist, and validated using gate-level simulation. Static Timing Analysis confirmed that the design met all timing constraints without setup or hold violations. The project demonstrates a complete FPGA design flow from RTL design to timing verification.

RTL Synthesis and Gate-Level Simulation of a Traffic Light Controller

References

1. Samir Palnitkar, Verilog HDL: A Guide to Digital Design and Synthesis, Prentice Hall.
2. M. Morris Mano and Michael D. Ciletti, Digital Design, Pearson Education.
3. Xilinx Vivado Design Suite User Guide (UG901).
4. Xilinx Vivado Design Suite User Guide: Logic Simulation (UG900).
5. IEEE Standard 1364-2005 Verilog Hardware Description Language.
6. Xilinx Documentation Portal.
7. FPGA4Student Verilog Design Examples.
8. NPTEL Digital System Design Course Materials